

db 성능 개선

```
Page<Product> findByTitleContaining(String title, Pageable pageable); 1개 사용 위치
```

GET http://localhost:8080/api/products/search?keyword=Handcrafted

Query Params

Key	Value	Description
keyword	Handcrafted	

Body

200 OK · 1.61 s · 479 B

JSON

```
@Query(value = "SELECT * FROM products WHERE MATCH(title) AGAINST(:keyword IN NATURAL LANGUAGE MODE)", 0개의 사용위치  
countQuery = "SELECT count(*) FROM products WHERE MATCH(title) AGAINST(:keyword IN NATURAL LANGUAGE MODE)",  
nativeQuery = true)  
Page<Product> searchByFullText(@Param("keyword") String keyword, Pageable pageable);
```

GET http://localhost:8080/api/products/search?keyword=Handcrafted

Query Params

Key	Value	Description
keyword	Handcrafted	

Body

200 OK · 7 ms · 479 B

- **B-Tree:** "데이터의 시작 부분부터 정렬하여 탐색 범위를 줄이는 방식" (접두어 검색에 유리)
- **Full-Text (Inverted Index):** "텍스트를 형태소 단위로 분리하여 각 단어가 포함된 위치를 매핑해 둔 방식" (키워드 포함 검색에 유리)

"단순한 **LIKE** 연산은 데이터가 늘어날수록 성능이 지수적으로 하락하지만, **Full-Text Index**는 생성 시점에 인덱싱 비용(Overhead)을 지불하는 대신, 검색 시점의 시간 복잡도를 **$O(1)$** 에 가깝게 낮추어 대규모 서비스의 확장성(Scalability)을 보장합니다.